FORMAL METHODS FOR SYSTEM VERIFICATION

Introduction to the course

Sabina Rossi

DAIS
Università Ca' Foscari
Venezia

Studying at Ca' Foscari

Master's Degree Programme in Computer Science and Information Technology [CM90]

- Degree Program Link: https://www.unive.it/degree/cm90
- Coordinator of the Degree Program Committee: Prof. Andrea Marin, marin@unive.it
- Student Services Office: campus.scientifico@unive.it

Studying at Ca' Foscari

Formal methods for system verification [CM0474]

- Link to the course Moodle space:
  https://moodle.unive.it/course/view.php?idnumber=323038
- Instructor:
  Prof. Sabina Rossi, sabina.rossi@unive.it

The exam can be taken in two ways

Assessment method 1

- A written test, with the possibility of an additional oral exam if deemed necessary by the professor or upon student request.
- The written test consists of open-ended questions on the course topics and exercises.
- The oral exam is conducted only if the written test is passed. In case of failure in the oral exam, the written test must be retaken.
- Dates :
  - Winter session (January-February): 2 exam dates
  - Summer session (May-June): 1 exam date
  - Autumn session (August-September): 1 exam date

The exam can be taken in two ways

Assessment method 2

- A project to be presented at the end of the lectures, followed by an oral exam.
- The oral exam includes a discussion of the project, solving exercises related to the project topic, and open-ended questions on the course subjects.
- Dates :
  - Beginning of November: the project is assigned.
  - December: project discussion and oral exam.

Key notions

- A model can be constructed to represent some aspects of the dynamic behaviour of a system.
- Once constructed, such a model becomes a tool with which we can investigate the behaviour of the system.

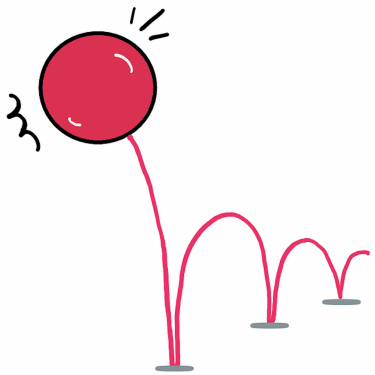
Descrete event systems

- In this course we will consider discrete event systems.
- What is a discrete event system ?
- A system whose state changes based upon the occurrence of discrete events.

Time driven vs. Event driven

Time driven: Bouncing ball

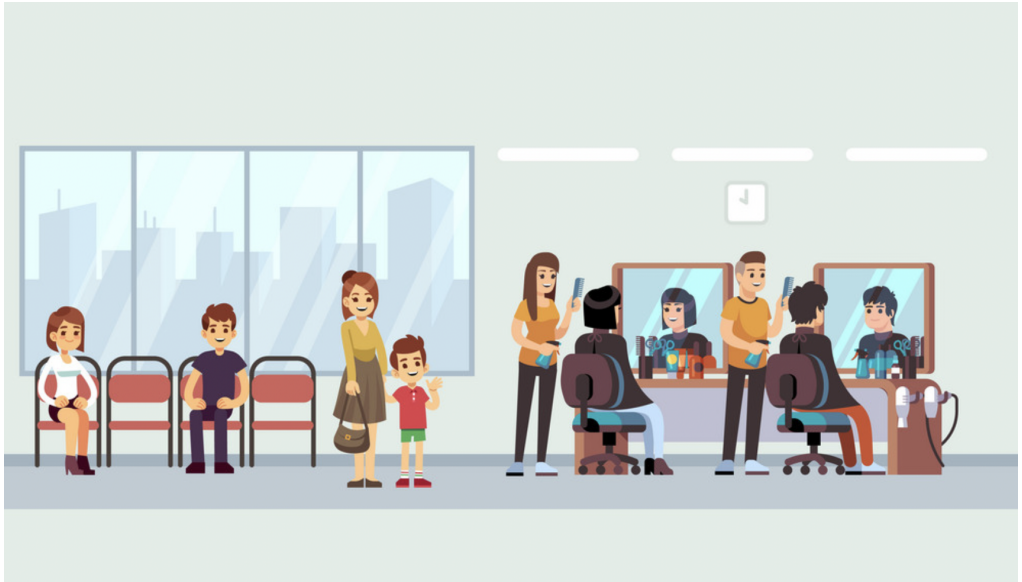
- State of the system is dependent on time and physical parameters:
  - initial height of ball
  - initial velocity of ball
  - gravity
- This is actually a deterministic system.
- The ball moves under the laws of physics (gravity, velocity,...).



Time driven vs. Event driven

Event driven: Barber shop

- This is an event driven system in which the state of the system does not change unless an event occurs:
  - customer arrives
  - customer queues
  - customer leaves
- This is actually a non-deterministic system.



Characterising a descrete event systems

- The state of the system is characterised by variables which take distinct values and which change by discrete events, i.e. at a distinct time something happens within the system which results in a change in one or more of the state variables.
- In the barber shop, the state evolves continuously with time, but occasionally the system undergoes discrete changes:
  - A new customer enters the system at a specific (possibly random) real time.
  - If the barber is busy, the customer joins the queue.
  - The barber finishes serving the current customer.

Example

- We might be interested in the number of nodes N in a communication network which are currently waiting to send a message
  - If a node, which was not previously waiting, generates a message and is now waiting to send then $N \to N + 1$.
  - If a node, which was previously waiting, successfully transmits its message then $N \to N - 1$.



Descrete event systems

Discrete time vs. Continuous time

- Within discrete event systems there is a distinction between a discrete time representation and a continuous time representation:

  Discrete time: such models only consider the system at predetermined moments in time, which are typically evenly spaced, e.g., at each clock "tick".

  Continuous time: such models consider the system at the time of each event so the time parameter in such models is conceptually continuous.

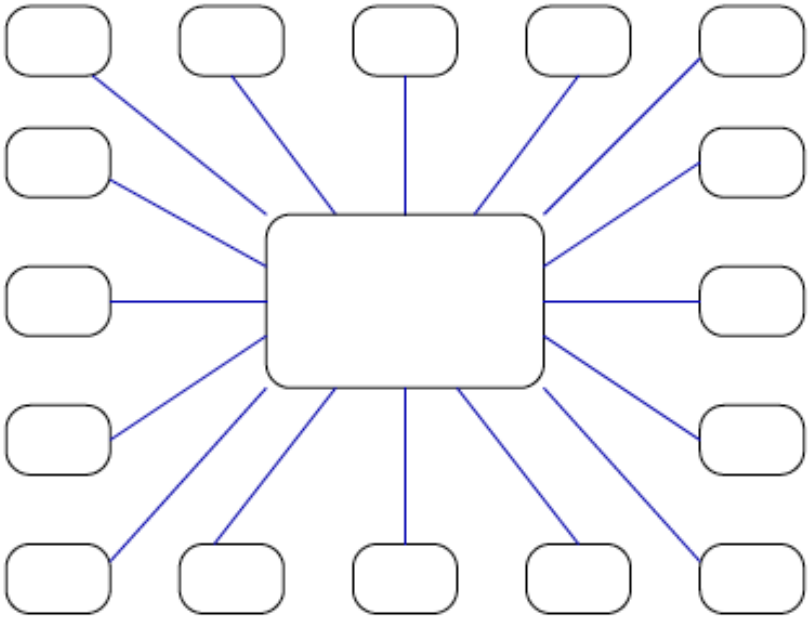
Continuous time models are generally appropriate for computer and communication systems.

Performance modelling

Modelling computer systems

- Performance modelling is concerned with the description, analysis and optimisation of the dynamic behaviour of computer and communication systems.
- The aim is to understand the behaviour of the system and identify the aspects of the system which are sensitive from a performance point of view.

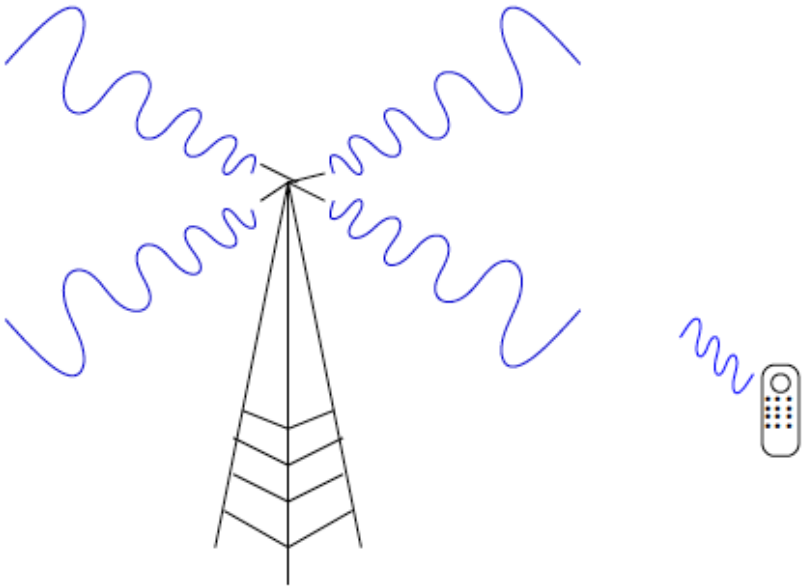
Performance Modelling: Motivation



Capacity planning

- How many clients can the existing server support and maintain reasonable response times?

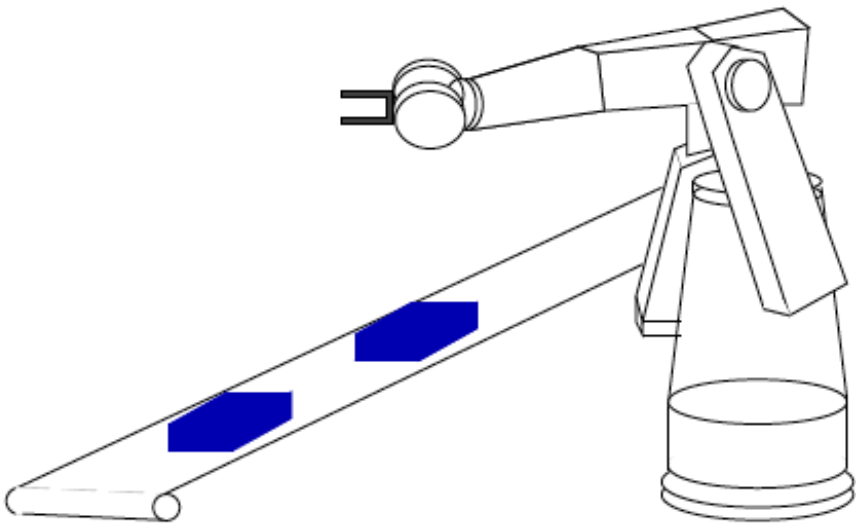
Performance Modelling: Motivation



System Configuration

- How many frequencies do you need to keep blocking probabilities low?

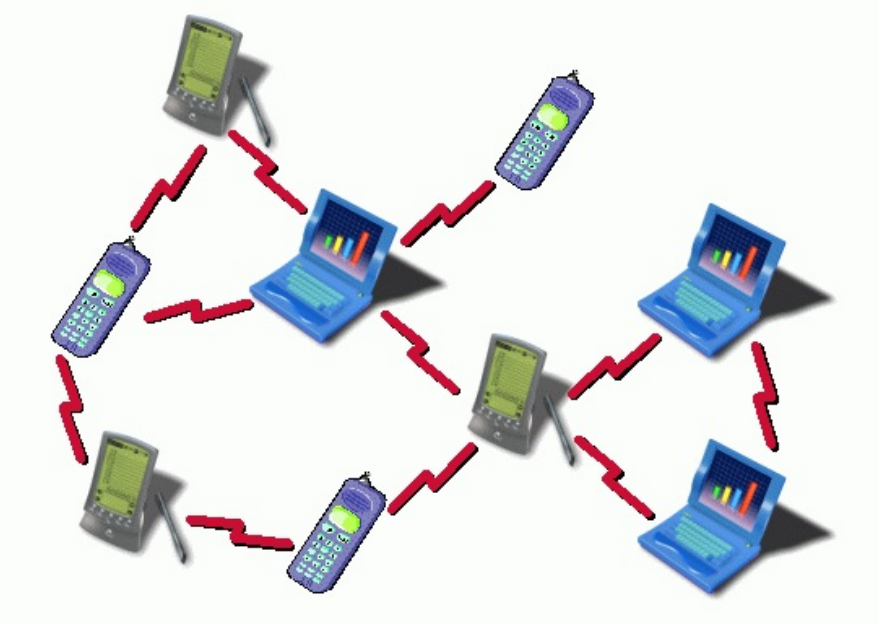
Performance Modelling: Motivation



System Tuning

- What speed of the conveyor belt will minimize robot idle time and maximize throughput whilst avoiding lost widgets?

Performance Modelling: Motivation



Sustainable energy planning

- Which is the minimum transmission radius to ensure connectivity while optimizing energy consumption?

Performance Modelling: Motivation



Prediction

- Which is the expected transaction consolidation time with respect to the offered fee?

Functional vs. Performance Modelling

Functional (Qualitative) Modelling

- Focus: What the system does.
- Describes the logical/functional behavior.
- Concerned with correctness: Does the system behave as intended? Does it meet specifications?

Performance (Quantitative) Modelling

- Focus: How well the system does it.
- Describes timing, probabilities, resource usage, throughput, delays.
- Concerned with efficiency: How fast? How reliable? How much resource is consumed?
- Uses stochastic models like Continuous-Time Markov Chains.

Performance Modelling

- It has been applied to computer systems since the mid-1960s and communication systems since the early 20th century.
- Originally queueing networks were primarily used to construct models, and sophisticated analysis techniques were developed.
- But as computer systems have developed these techniques are no longer widely applicable for expressing the dynamic behaviour observed in distributed systems.

Process Algebras

- Process algebras are mathematical theories which model concurrent systems by their algebra and provide apparatus for reasoning about the structure and behaviour of the model.
- Process algebras where models are decorated with quantitative information used to generate a stochastic process are called stochastic process algebras (SPA).

Process Algebras

- Examples of process algebras include:
  - CCS - Calculus of Communicating Systems
  - CSP - Communicating Sequential Processes
  - ACP - Algebra of Communicating Processes
  - PEPA - Performance Evaluation Process Algebra

Probabilistic/Stochastic Process Algebras

- Classical process algebras are used to model systems in which there is uncertainty about the behaviour of a component but, like time, this uncertainty is abstracted away so that all choices become nondeterministic.
- Probabilistic extensions of process algebras allow this uncertainty to be quantified because nondeterministic choice is replaced by a probabilistic choice.
- In this case a probability is associated with each possible outcome of a choice.

The "Cooperator" Paradigm

- A system is characterised by its active components and the interactions, or communications, or cooperation between them.
- For example, in distributed systems and communication networks components have autonomy and the framework is the one of cooperation.
- In a process algebra model all system elements have equal status; the model defines their individual behaviours and how they interact.

Hierarchical Models

- Process algebras include mechanisms for compositional reasoning.
- These mechanisms, which are an integral part of the language, facilitate the systematic development of large models with hierarchical structure.

Process Algebras as a Design Methodology

- The process algebra style of system description is close to the way that designers think about systems.
- Using a process algebra based language for performance modelling introduces the possibility of a closer integration of performance analysis into design methodologies.
- It allows to perform both qualitative (or functional) and quantitative modelling using the same system description.

The PEPA project

The PEPA project started in Edinburgh in 1991.

Motivation: the size and complexity of many modern systems result in large, complex models.

Idea: A compositional approach decomposes the system into subsystems that are smaller and more easily modelled.

Objective: The PEPA project aims at providing a novel compositional approach to performance modelling.

Result: PEPA is a stochastic process algebra.

References

- PEPA:

  http://www.dcs.ed.ac.uk/pepa/

- Textbook: J. Hillston. A Compositional Approach to Performance Modelling. Cambridge University Press, 1996.

  http://www.dcs.ed.ac.uk/pepa/papers/